



LISTA DE EXERCÍCIOS - SISTEMAS INTELIGENTES APLICADOS

Lista de Exercícios 9

KNN

DOCENTE: Thiago França Naves

DATA: 30/09/2026

ALUNO: Mayara Rodrigues Pereira e Leonardo José Reis Pinto

A questão 1 foi feita no Weka 3.8.6 com o IBk e validação cruzada de 10 dobras. As questões 2 a 4 estão no script `solucao.py` (Apêndice A), que implementa o KNN com as distâncias euclidiana e Manhattan.

Questão 1

No Weka na aba de Classificadores, pasta de algoritmos Lazy selecione o IBk que é a implementação do KNN. Faça testes com o algoritmo com a base Iris e Weather, execute testes com o algoritmo e teste diferentes valores de K, por fim discorra sobre os resultados, acurácia e matriz de confusão.

Resposta. O IBk não monta modelo no treino, só guarda os exemplos. Para classificar, ele calcula a distância do exemplo novo a todos os guardados, pega os K mais próximos e devolve a classe mais votada. Os atributos numéricos são normalizados para a faixa de 0 a 1, e nos nominais a distância é 0 se os valores forem iguais e 1 se forem diferentes. Usamos a `iris.arff` (150 flores, 4 medidas, 3 espécies com 50 cada) e a `weather.nominal.arff` (14 dias, 4 atributos nominais), variando o campo KNN.

Tabela 1. IBk com validação cruzada de 10 dobras.

Iris			Weather		
K	Acurácia	Kappa	K	Acurácia	Kappa
1	95,33%	0,93	1	57,14%	0,067
3	95,33%	0,93	3	64,29%	0,103
5	95,33%	0,93	5	64,29%	0
7	96,67%	0,95	7	64,29%	0
9	96,00%	0,94	9	64,29%	0
15	96,67%	0,95	15	64,29%	0

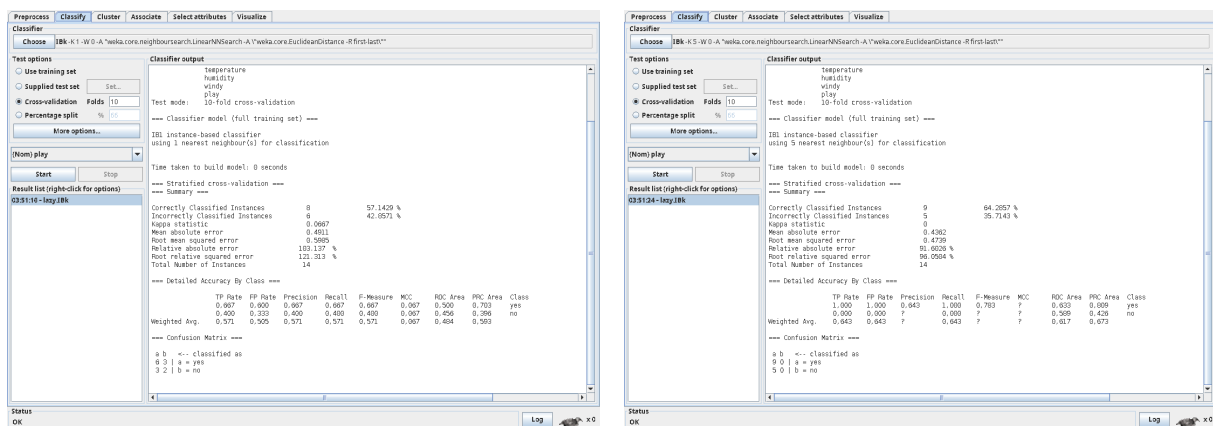
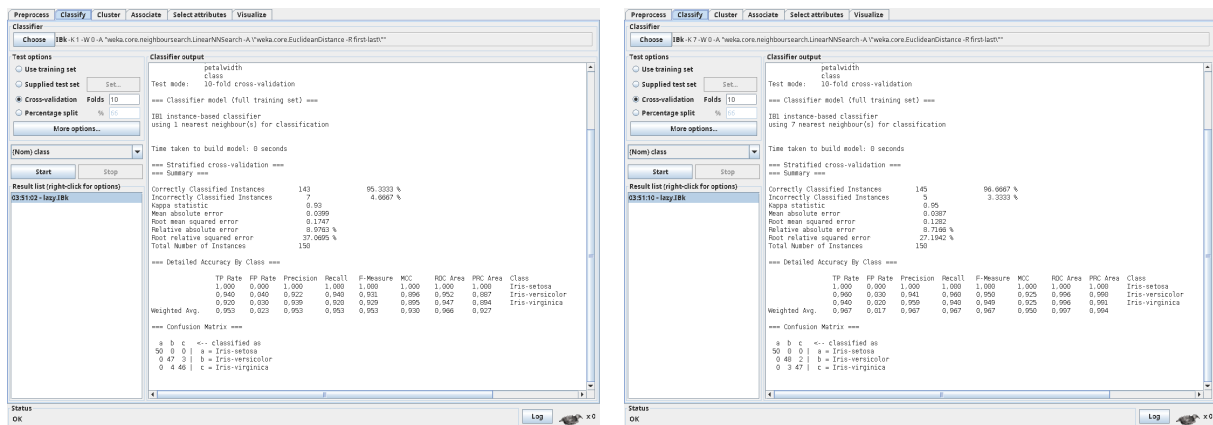


Figura 2. IBK na Weather com K = 1 (esquerda) e K = 5 (direita).

Tabela 2. Matrizes de confusão (linhas: classe real).

Iris, K = 1				Iris, K = 7			
	set.	vers.	virg.		set.	vers.	virg.
setosa	50	0	0	setosa	50	0	0
versicolor	0	47	3	versicolor	0	48	2
virginica	0	4	46	virginica	0	3	47
Weather, K = 1				Weather, K = 5			
	yes	no			yes	no	
yes	6	3		yes	9	0	
no	3	2		no	5	0	

Na **Iris** o KNN vai muito bem com qualquer K, entre 95% e 97%. A matriz mostra que a setosa nunca é confundida: ela fica separada das outras no espaço das medidas das pétalas. Todos os erros são trocas entre versicolor e virginica, que se sobrepõem numa faixa de valores. Com K = 1 são 7 erros, e com K = 7 caem para 5. Um K maior suaviza a fronteira nessa região de sobreposição, porque uma flor atípica deixa de decidir sozinha a classe do vizinho.

Na **Weather** o resultado é ruim com qualquer K. Com K = 1 ele acerta 8 de 14 (57%), abaixo de chutar sempre “yes”. A partir de K = 5 a acurácia sobe para 64,29%, mas a matriz mostra por quê: tudo é classificado como “yes”, e nenhum “no” é reconhecido (kappa 0). Com 12 ou 13 dias de treino por dobra, 5 vizinhos já são quase metade da base, e a maioria “yes” ganha sempre. Com atributos nominais, a distância só conta quantos valores são diferentes, e muitos dias ficam empatados na mesma distância. A base é pequena demais para o KNN, que depende de ter vizinhos parecidos de verdade.

Comparando as duas: K pequeno segue os detalhes e é sensível a ruído, K grande suaviza, mas quando fica grande demais em relação à base o KNN vira um classificador de maioria. Na Iris, com 150 exemplos, K entre 7 e 15 foi o melhor. Na Weather nenhum K resolveu.

Questão 2

Supondo que desejamos classificar visualmente os 5 principais personagens do desenho animado “Os Simpsons”. a. Com base na imagem abaixo, selecione um conjunto de pelo menos 5 características numéricas para serem utilizadas no algoritmo KNN. b. Utilizando as características escolhidas, crie um conjunto de treinamento (com pelo menos 10 exemplos), utilizando os atributos definidos no exercício anterior. c. Crie um conjunto de treinamento com 3 novos exemplos e execute o KNN para classificar estes.

Resposta. a) Escolhemos características que dá para medir na figura e que diferenciam os personagens:

1. **altura**, medida em relação à altura do Homer (a Marge fica acima de 1 por causa do cabelo, a Maggie perto de 0,35);
2. **pontas no cabelo**: 9 no Bart, 8 na Lisa, 1 na Maggie, 2 fios no Homer e nenhuma na Marge;
3. **fração de azul** na figura: o cabelo da Marge, o macacão da Maggie, as calças do Homer, o short do Bart;
4. **fração de vermelho ou laranja**: o vestido da Lisa e a camiseta do Bart;
5. **fração de branco**: a camisa do Homer e o colar da Lisa.

As três primeiras separam os personagens por forma e as duas últimas por cor. Bart e Lisa são parecidos em altura e cabelo e se separam pelo azul do short do Bart.

b) Montamos 15 exemplos, 3 por personagem, imaginando o personagem em poses e desenhos um pouco diferentes, com pequenas variações nas medidas:

Tabela 3. Conjunto de treinamento.

Ex.	Alt.	Pontas	Azul	Verm.	Branco	Ex.	Alt.	Pontas	Azul	Verm.	Branco
Homer 1	1,00	2	0,25	0,00	0,30	Bart 3	0,65	8	0,10	0,33	0,04
Homer 2	0,98	2	0,28	0,00	0,27	Lisa 1	0,58	8	0,00	0,40	0,05
Homer 3	1,03	1	0,22	0,02	0,32	Lisa 2	0,55	8	0,00	0,43	0,06
Marge 1	1,25	0	0,35	0,03	0,00	Lisa 3	0,60	7	0,02	0,38	0,04
Marge 2	1,20	0	0,32	0,02	0,02	Maggie 1	0,35	1	0,45	0,05	0,02
Marge 3	1,30	0	0,38	0,04	0,00	Maggie 2	0,33	1	0,48	0,06	0,03
Bart 1	0,62	9	0,12	0,30	0,03	Maggie 3	0,38	1	0,42	0,04	0,02
Bart 2	0,60	9	0,15	0,28	0,02						

c) Criamos 3 exemplos novos: $n1 = (0,57; 8; 0,03; 0,36; 0,05)$, $n2 = (1,02; 2; 0,20; 0,01; 0,35)$ e $n3 = (0,40; 1; 0,40; 0,07; 0,03)$. As escalas são muito diferentes (as pontas vão até 9 e as frações até 0,5), então normalizamos cada atributo para 0 a 1 com o mínimo e o máximo do treino antes de calcular a distância euclidiana. Sem isso, o número de pontas dominaria a distância.

Tabela 4. Classificação dos exemplos novos.

Exemplo	K = 1 (vizinho e distância)	K = 3
n1	Lisa (Lisa 1, 0,113)	Lisa (Lisa 1, Lisa 3, Lisa 2)
n2	Homer (Homer 3, 0,153)	Homer (Homer 3, Homer 1, Homer 2)
n3	Maggie (Maggie 3, 0,089)	Maggie (Maggie 3, Maggie 1, Maggie 2)

Os três foram classificados como o personagem que imaginamos, com K = 1 e com K = 3. Isso

era esperado, porque escolhemos as medidas por separar bem os personagens. O caso mais difícil seria entre Bart e Lisa, e o que os separa aqui é a fração de azul.

Questão 3

A tabela abaixo representa um conjunto de treino para classificação de livros em duas categorias, Class L = romance e Classe C = crimes. [...] normalize os valores do conjunto de treino e utilize o KNN para classificar os novos livros d6 e d7 em suas respectivas categorias. Utilize duas medidas de distância diferentes.

Resposta. O treino tem d1, d2 e d3 (romance) e d4 e d5 (crime). Normalizamos cada termo com $x' = (x - \min) / (\max - \min)$, usando o mínimo e o máximo do treino, e aplicamos a mesma conta nos livros novos. O “murderer” vai até 56 e dominaria a distância se não fosse normalizado.

Tabela 5. Valores normalizados (love, kiss, inspector, murderer).

Livro	love	kiss	inspector	murderer
d1 (romance)	1,000	0,833	0,167	0,000
d2 (romance)	0,800	1,000	0,000	0,018
d3 (romance)	0,700	0,667	0,000	0,000
d4 (crime)	0,000	0,167	1,000	0,357
d5 (crime)	0,100	0,000	0,667	1,000
d6	0,500	1,000	0,167	0,000
d7	0,100	0,000	1,000	0,071

Usamos a distância euclidiana, $\sqrt{\sum (a_i - b_i)^2}$, e a Manhattan, $\sum |a_i - b_i|$.

Tabela 6. Distâncias dos livros novos até cada livro de treino.

	Distância	d1	d2	d3	d4	d5
d6	euclidiana	0,527	0,344	0,423	1,329	1,552
d6	Manhattan	0,667	0,485	0,700	2,524	2,900
d7	euclidiana	1,485	1,579	1,345	0,346	0,987
d7	Manhattan	2,638	2,754	2,338	0,552	1,262

Com as duas distâncias, e com $K = 1$ ou $K = 3$, **d6 é romance** (o mais próximo é d2, e os três mais próximos são os três romances) e **d7 é crime** (o mais próximo é d4, e entre os três mais próximos há dois crimes). O d6 tem muito “kiss” e “love” e quase nada das palavras de crime, e o d7 é o contrário, com 12 páginas de “inspector”. As duas distâncias deram a mesma ordem de vizinhos, o que mostra que a classificação é segura.

Questão 4

Considere a imagem abaixo e os pontos nela descritos. Utilize o KNN com valores de $K = 1$ e $K = 3$ e classifique um novo ponto na posição (8,1).

Resposta. Lendo a figura, a classe * tem os pontos (2,4), (4,6), (6,4) e (4,2), e a classe + tem (4,4) e (6,2). As distâncias euclidianas até (8,1) são:

Ponto	Classe	Distância até (8,1)
(6,2)	+	$\sqrt{4+1} = 2,236$
(6,4)	*	$\sqrt{4+9} = 3,606$
(4,2)	*	$\sqrt{16+1} = 4,123$
(4,4)	+	$\sqrt{16+9} = 5,000$
(4,6)	*	$\sqrt{16+25} = 6,403$
(2,4)	*	$\sqrt{36+9} = 6,708$

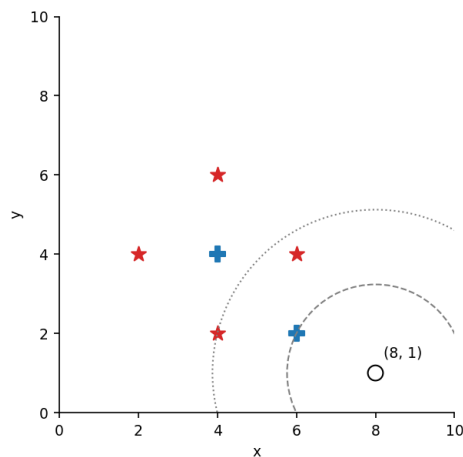


Figura 3. O novo ponto (círculo vazio). O círculo tracejado alcança o vizinho mais próximo e o pontilhado alcança os três mais próximos.

Com $K = 1$ o vizinho é (6,2), e o ponto é classificado como +. Com $K = 3$ os vizinhos são (6,2) +, (6,4) * e (4,2) *, e a votação dá 2 a 1 para *. O resultado muda com K porque o ponto está fora da nuvem de dados, e o único + próximo perde a votação quando entram mais vizinhos. Nesse caso nenhuma das duas respostas é muito confiável, já que o vizinho mais próximo está a mais de duas unidades.

Apêndice A. Código

```
pip install matplotlib
python solucao.py

# Lista 9 - KNN
# Sistemas Inteligentes Aplicados - UTFPR
# Uso: python solucao.py

import os
from collections import Counter
from math import sqrt
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

PASTA = os.path.dirname(os.path.abspath(__file__))
SAIDA = os.path.join(PASTA, "saidas")
os.makedirs(SAIDA, exist_ok=True)
plt.rcParams.update({"font.size": 9, "axes.spines.top": False,
                    "axes.spines.right": False})

def euclidiana(a, b):
    return sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def manhattan(a, b):
    return sum(abs(x - y) for x, y in zip(a, b))

def knn(treino, x, k, dist):
    # treino: lista de (nome, vetor, classe)
    ordem = sorted(treino, key=lambda t: dist(t[1], x))
    vizinhos = ordem[:k]
    votos = Counter(c for _, _, c in vizinhos)
    return votos.most_common(1)[0][0], vizinhos

def normalizar(treino, novos):
    # min-max com os limites do treino
    n = len(treino[0][1])
    minimo = [min(t[1][i] for t in treino) for i in range(n)]
    maximo = [max(t[1][i] for t in treino) for i in range(n)]

    def escala(v):
        return [(v[i] - minimo[i]) / (maximo[i] - minimo[i]) for i in range(n)]

    t2 = [(nome, escala(v), c) for nome, v, c in treino]
    n2 = [(nome, escala(v), c) for nome, v, c in novos]
    return t2, n2

# questao 2: Simpsons
# altura (em cabeças de altura do Homer), pontas no cabelo, e a fração da
# figura que é azul, vermelho/laranja e branca, medidas a olho na imagem
print("[2] Simpsons")
simpsons = [
    ("h1", [1.00, 2, 0.25, 0.00, 0.30], "Homer"),
    ("h2", [0.98, 2, 0.28, 0.00, 0.27], "Homer"),
    ("h3", [1.03, 1, 0.22, 0.02, 0.32], "Homer"),
    ("m1", [1.25, 0, 0.35, 0.03, 0.00], "Marge"),
    ("m2", [1.20, 0, 0.32, 0.02, 0.02], "Marge"),
    ("m3", [1.30, 0, 0.38, 0.04, 0.00], "Marge"),
    ("b1", [0.62, 9, 0.12, 0.30, 0.03], "Bart"),
    ("b2", [0.60, 9, 0.15, 0.28, 0.02], "Bart"),
    ("b3", [0.65, 8, 0.10, 0.33, 0.04], "Bart"),
    ("l1", [0.58, 8, 0.00, 0.40, 0.05], "Lisa"),
    ("l2", [0.55, 8, 0.00, 0.43, 0.06], "Lisa"),
    ("l3", [0.60, 7, 0.02, 0.38, 0.04], "Lisa"),
    ("g1", [0.35, 1, 0.45, 0.05, 0.02], "Maggie"),
    ("g2", [0.33, 1, 0.48, 0.06, 0.03], "Maggie"),
    ("g3", [0.38, 1, 0.42, 0.04, 0.02], "Maggie"),
```

```

]
novos = [
    ("n1", [0.57, 8, 0.03, 0.36, 0.05], "Lisa"),
    ("n2", [1.02, 2, 0.20, 0.01, 0.35], "Homer"),
    ("n3", [0.40, 1, 0.40, 0.07, 0.03], "Maggie"),
]
treino_n, novos_n = normalizar(simpsons, novos)
for k in [1, 3]:
    for nome, v, real in novos_n:
        pred, viz = knn(treino_n, v, k, euclidiana)
        dists = ", ".join(f"{n}:{euclidiana(x, v):.3f}" for n, x, _ in viz)
        print(f" k={k} {nome}: previsto={pred} (real {real}) vizinhos {dists}")

# questao 3: livros
print("\n[3] livros")
livros = [("d1", [10, 5, 2, 0], "romance"), ("d2", [8, 6, 0, 1], "romance"),
          ("d3", [7, 4, 0, 0], "romance"), ("d4", [0, 1, 12, 20], "crime"),
          ("d5", [1, 0, 8, 56], "crime")]
novos = [("d6", [5, 6, 2, 0], "?"), ("d7", [1, 0, 12, 4], "?")]
livros_n, novos_n = normalizar(livros, novos)
for nome, v, _ in livros_n + novos_n:
    print(f" {nome}: " + " ".join(f"{x:.3f}" for x in v))
for dist in [euclidiana, manhattan]:
    print(f" distância {dist.__name__}:")
    for nome, v, _ in novos_n:
        ds = ", ".join(f"{n}={dist(x, v):.3f}" for n, x, _ in livros_n)
        p1, _ = knn(livros_n, v, 1, dist)
        p3, _ = knn(livros_n, v, 3, dist)
        print(f" {nome}: {ds} -> k=1 {p1}, k=3 {p3}")

# questao 4: ponto (8,1)
print("\n[4] ponto (8, 1)")
pontos = [("A", [2, 4], "*"), ("B", [4, 6], "*"), ("C", [6, 4], "*"),
          ("D", [4, 2], "*"), ("E", [4, 4], "+"), ("F", [6, 2], "+")]
novo = [8, 1]
for nome, p, c in sorted(pontos, key=lambda t: euclidiana(t[1], novo)):
    print(f" {p} {c}: distância {euclidiana(p, novo):.3f}")
for k in [1, 3]:
    pred, _ = knn(pontos, novo, k, euclidiana)
    print(f" k={k}: {pred}")

fig, ax = plt.subplots(figsize=(4.2, 4.2))
for _, (x, y), c in pontos:
    ax.scatter(x, y, s=90, marker="*" if c == "*" else "p",
              color="tab:red" if c == "*" else "tab:blue")
ax.scatter(*novo, s=90, marker="o", facecolor="none", edgecolor="k")
ax.annotate("(8, 1)", (8, 1), (8.2, 1.4))
for raio, estilo in [(euclidiana([6, 2], novo), "--"),
                     (euclidiana([4, 2], novo), ":")]:
    ax.add_patch(plt.Circle(novo, raio, fill=False, ls=estilo, color="gray"))
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)
ax.set_aspect("equal")
ax.set_xlabel("x")
ax.set_ylabel("y")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q4_pontos.png"), dpi=150)
plt.close(fig)

```